✳ 브로드 캐스팅 규칙.

브로드 캐스팅: 형태가 다른 배열끼리 연산할 때 Numpy가 작은 쪽을 큰 쪽에 맞추어 확장하는 기능.

적용 규칙 1. 뒤에서부터 하나씩 차원 비교.
2. 같은 형태 or (n,1) or (1,n) 형태이면 통과.
3. 차원 수가 다르면 앞쪽에 1을 채워 맞춤.

Ex) 기본 전제 조건: $n \neq m$, $n > 1$, $m > 1$

I. (n,m), 스칼라 (0차원) 연산 → 결과: (n,m)

```
a = np.arange(1,7).reshape(2,3)   # [1,2,3,4,5,6] → [ [1,2,3], [4,5,6] ]

b = a + 10                        # 10 : ( ) → (2,3)

# numpy가 스칼라 10을 [ [10,10,10], [10,10,10] ] 으로 확장하여 연산 수행

print(b)                          # [ [11, 12, 13], [14, 15, 16] ]
```

II. (n,m), (m,) 연산 → 결과: (n,m)

```
a,b = np.ones((2,3), dtype=int), np.ones( (3, ), dtype=int)      # b = [1,1,1]

print(a+b)                  # [ [2,2,2], [2,2,2] ]
print( a+b, (a+b).shape)    # (2,3)
```

III. (n,m), (n,) 연산 → 결과: ValueError ★ 뒤에서부터 비교하므로 뒷부분 크기가 같아야 연산 가능.

뒤에서부터 비교하므로 (1,n)으로 취급되는데, 이러면 열 크기가 각각 m, n 이다.
모든 예시의 기본 전제가 $n \neq m$이고 n과 m은 1이 아니므로, 적용 규칙 2의 통과조건을 충족하지 못한다.
크기도 다르고, 둘 중 어느 것도 1이 아님. ↵

⇒ ValueError: operands could not be broadcast together with shapes (2,3) (2,)

IV. (n,m), (n,1) 연산
(n,m), (1,m) 연산 → 결과 : (2,3)

```
a, b = np.ones( (2,3), dtype=int), np.ones( (2, 1), dtype=int)

print(a+b)                  # [ [2,2,2], [2,2,2] ]

# 3 vs 1 → 크기는 다르지만 둘 중 하나가 1이므로 통과.

print( (a+b).shape)         # (2,3)

a, b = np.ones( (2,3), dtype=int), np.ones( (1,3), dtype=int)

print(a+b)                  # [ [2,2,2], [2,2,2] ]

# 2 vs 1 → 크기는 다르지만 둘 중 하나가 1이므로 통과.

print( (a+b).shape)         # (2,3)
```

V. (n, m) 에 .mean(axis=1) 적용, (n,m) 연산 → 결과 : ValueError

```
a = [[1,2,3], [4,5,6]]          # (2,3)
a_mean = a.mean(axis=1, dtype=int)
# a_mean = [2, 5], shape = (2, ) a에 있던 1번 축(3)이 사라져있음.

b = np.ones((2,3), dtype=int)

print(a_mean + b)      # (2,) (2,3)을 연산하게 되어 표과 같이 ValueError 발생
```

VI. (n,m)에 .mean(axis=1, keepdims=True) 적용, (n,m) 연산 → 결과 : (n,m)

```
a = [[1,2,3], [4,5,6]].mean(axis=1, dtype=int, keepdims=True)
# a = [[2], [5]] , a.shape = (2,1)

b = np.ones((2,3), dtype=int)

print(a+b)      # (2,1) 형태였던 a가 b의 (2,3) 형태에 맞게 브로드캐스팅 됨.
# [[2,2,2], [5,5,5]] + [[1,1,1], [1,1,1]] ⇒ 결과 : [[3,3,3], [6,6,6]]
```

★ 브로드 캐스팅 규칙은 Pandas의 DataFrame에서도 동일하게 적용됨!

```
df = pd.DataFrame({"A": [1,3,5], "B": [2,4,6]}, index=["r1","r2","r3"])  # (3,2)
s = pd.Series([10,20], index=["B","A"])  # A열에 20, B열에 10. (2,) 형태.

result = df+s
print(result)
"""
        A       B       s = [20,10] (A열부터) → [[20,10], [20,10], [20,10]]
   r1   21      12       (df의 shape에 맞게 (3,2)로 확장.)
   r2   23      14
   r3   25      16       위치가 아닌 컬럼 라벨로 매칭하여 연산 수행.
"""
```

동일한 인덱스가 없을 경우 : 연산 결과가 NaN (결측)★으로 채워짐.

```
df = pd.DataFrame({"A": [1,3,5], "B": [2,4,6]}, index=["r1","r2","r3"])
s = pd.Series([10,20], index=["A","C"])

print(df+s)
"""
        A     B     C
   r1   11.0  NaN   NaN       A열 : NaN이 섞여서 float으로 자동 변환됨.
   r2   13.0  NaN   NaN       B열 : 시리즈에 B열이 부재 → NaN
   r3   15.0  NaN   NaN       C열 : 데이터 프레임에 C열 부재 → NaN
"""
```

✳ DataFrame 인덱싱 메커니즘 종류와 주의사항

1. 기본 인덱싱 데이터프레임명 ['컬럼명'] 가장 원초적인 방식.

1) 종류/문법

단일 컬럼 선택: df["Pclass"] # 'Pclass'열 선택. (Series 반환)
df[['Pclass']] # 열 1개짜리 DataFrame 반환.
원본 df의 형태가 (n,m)이었으면 (n,1)

복수 컬럼 선택: df["Pclass", "Fare"] # 'Pclass'열, 'Fare'열 선택. (DataFrame 반환)

2) 주의 사항

컬럼명이 정수형이 아닐 때: df[0] 처럼 작성시 KeyError 발생.
(이유는 간단하다. 0이라는 라벨을 가진 컬럼이 없기 때문.)

df["행이름"]과 같은 문법은 없다. KeyError 발생.

슬라이싱 df[0:2]: 컬럼 슬라이싱을 해주는 게 아니라 0행부터 1행까지 슬라이싱 됨.
(원본 shape이 (n,m)이었다면 (2,m) DataFrame 반환)

2. 라벨 인덱싱 loc (Label Location)

예시 데이터: df = pd.DataFrame({"Age": [41, 22],
"Sword": ["Rebellion", "RedQueen"],
"Gun": ["Dr.Faust", "BlueRose"]},
index=["Dante", "Nero"])

	Age	Sword	Gun
Dante	41	Rebellion	Dr.Faust
Nero	22	RedQueen	BlueRose

1) 종류/문법

특정 행만 선택 df명.loc["행이름"]

print(df.loc["Dante"]) # Age: 41, Sword: Rebellion, Gun: Dr.Faust (Series 형태)

특정 열만 선택 df명.loc[:, "열이름"] (df명["열이름"]이 더 자주 쓰임.)

print(df.loc["Age"]) # Series 형태 반환

특정 셀 선택 df명.loc["행이름", "열이름"]

print(df.loc["Nero", "Gun"]) # 단일값 "BlueRose"

슬라이싱 df명.loc["시작행이름":"끝행이름", "시작열이름":"끝열이름"]

print(df.loc["Dante":"Nero", "Age":"Gun"]) # 끝값이 포함되므로 원본 df와 동일.

불리언 마스킹과 함께 쓰기 df명.loc[조건식, ["열이름1", "열이름2"]]

print(df.loc[(df["Age"] > 30) & (df["Sword"].str.len == 9), ["Age", "Sword"]])

```
""
            Age    Sword      ↳ Age 컬럼 값이 30 초과 and Sword 컬럼값의 글자수가 9인
Dante       41     Rebellion     행의 Age, Sword 컬럼만 추출
""
```

2) 주의 사항

슬라이싱 시에 끝값이 포함된다.

df.loc[, "열이름"] 같은 문법은 없다. SyntaxError 발생.

3. 위치 인덱싱 iloc (Integer Location)

1) 종류/문법 (예시 데이터는 loc와 동일.)

특정 행만 선택 df명.iloc[행번호] — dante = df.iloc[0]

특정 열만 선택 df명.iloc[:, 열번호] — guns = df.iloc[:, 2]

특정 셀 선택 df명.iloc[행번호, 열번호] — Nero_s_age = df.iloc[1, 0]

행·열 슬라이싱 df명.iloc[시작행번호 : 끝행번호, 시작열번호 : 끝열번호]

df2 = df.iloc[0:1, 0:2] # 0번째 행의 0, 1열만 선택됨. loc와 달리 끝값은 제외.

2) 주의사항

슬라이싱 시에 끝값은 제외된다.

불린언 마스킹을 적용할 수 없다.

(df.iloc[(df["Age"] >40).values] 형태는 가능하나, df[df["Age"] >40]이 더 쉽다.)

4. 불린언 인덱싱

1) 종류/문법

기본 인덱싱에 적용 df명[(조건식1) & (조건식2)] (& 또는 | (ㅠ)) — df[df["Age"] < 25]

loc에 적용 df명.loc[조건식, "열이름"] — df.loc[df["Age"] == 41, "Sword"]

2) 주의사항

복수 조건식을 적용할 경우 각 조건을 필히 소괄호 ()로 감싸야 한다.

감싸지 않으면 연산자 우선순위로 인해 &와 | (ㅠ)가 먼저 인식되어 ValueError가 발생한다.

(만에 하나 오류가 발생하지 않더라도 각 조건을 제대로 읽지 못해 엉뚱한 결과가 나온다.)

여러 개의 조건식 사이에 &, | (ㅠ) 대신 and, or을 작성하면 ValueError가 발생한다.

5. 단일값(스칼라) 추출 .at[ ] / .iat[ ]

1) 종류/문법

라벨기반 df명.at["행이름", "열이름"] — df.at["Dante", "Gun"] # Dr. Faust

위치기반 df명.iat[행번호, 열번호] — df.iat[1, 2] # BlueRose

2) 주의사항

슬라이싱 불가. 여러 값 동시추출 불가.